

UNIVERSITÀ DEGLI STUDI DI NAPOLI FEDERICO II

SCUOLA POLITECNICA E DELLE SCIENZE DI BASE

DIPARTIMENTO DI INGEGNERIA ELETTRICA E TECNOLOGIE DELL'INFORMAZIONE



Elaborato di Network & Cloud Infrastructures

SDN DoS Detection & Mitigation con Mininet e Ryu

Anno Accademico 2025/2026

Prof. Giorgio Ventre

Studente:

Accadia Mattia - M63001600

Introduzione

L'obiettivo del progetto è simulare una rete SDN tramite l'emulatore Mininet e il controller Ryu, implementando un sistema di monitoraggio in grado di rilevare e bloccare in tempo reale un attacco DoS proveniente da uno degli host della topologia.

In una SDN l'intelligenza della rete viene spostata nel piano di controllo (control plane), reso programmabile via software. Il controller comunica con gli switch tramite il protocollo OpenFlow, adottato sia dagli switch software OpenvSwitch sia dal controller Ryu.

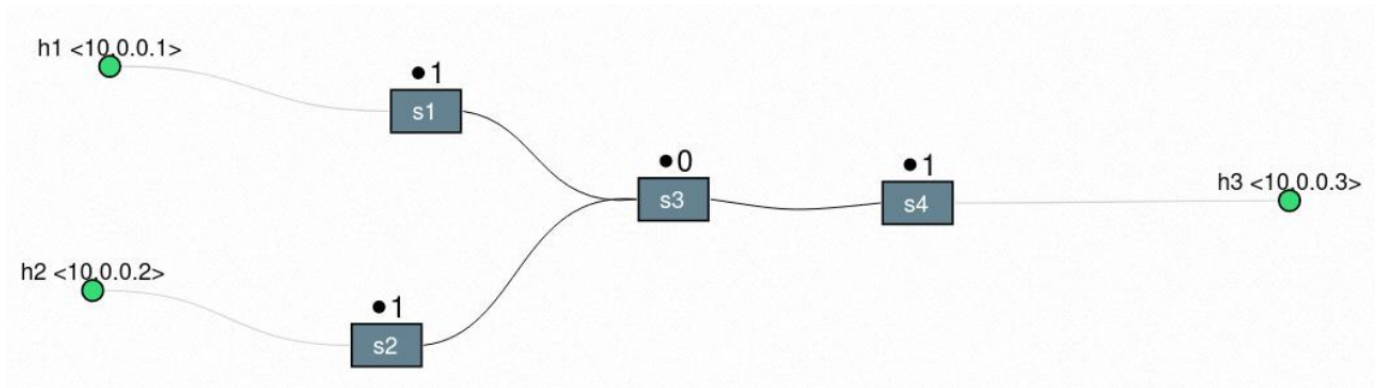
Topologia

La topologia di rete che si intende simulare si compone di tre host, quattro OpenvSwitch e un controller Ryu che coordini le operazioni degli switch. In una SDN, infatti, l'intelligenza della rete viene spostata nel controllore, programmabile tramite software, consentendo pertanto flessibilità nella gestione di una architettura di rete che diviene sempre più complessa. Il controller opera a livello control plane e utilizza un open protocol per ricevere informazioni di stato dagli elementi di forwarding e per fornire ad essi controlli e direttive. In particolare, il protocollo più utilizzato a tale scopo è OpenFlow, adottato dai particolari switch software che si è scelto di utilizzare, gli OpenvSwitch e dal controller Ryu.

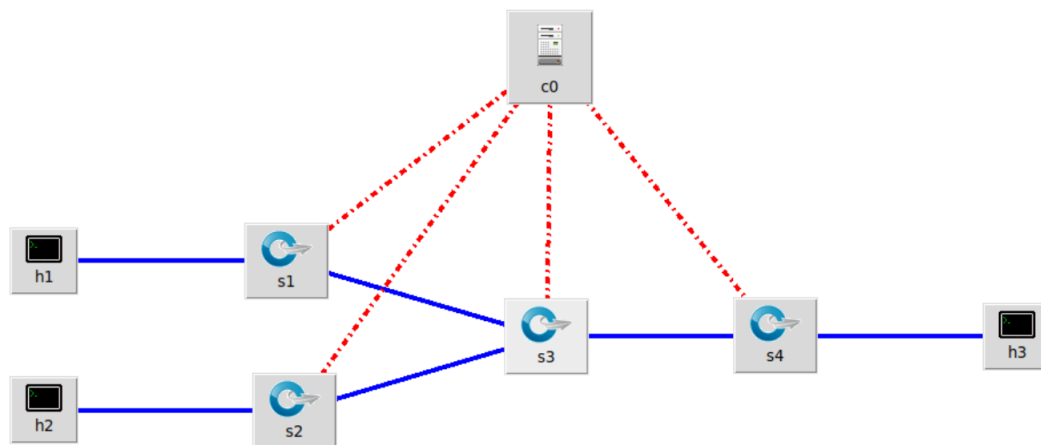
Si noti che il controller istanziato appartiene alla classe `RemoteController` e sarà dunque eseguito da linea di comando prima di lanciare lo script contenente la topologia così da risultare disponibile, gli switch istanziati sono di tipo OpenvSwitch, come determina l'opzione `cls=OVSKernelSwitch` e la larghezza di banda è settata tenendo conto del differente carico a cui sono soggetti i link che connettono un host client a uno switch rispetto ai link che connettono più switch nel backbone della rete e uno switch a un host che funge da server, i quali convogliano più flussi di dati provenienti da diversi end-system e hanno pertanto necessità di avere una capacità maggiore.

Una volta costruita e avviata la rete con i comandi `build()` e `start()`, si esegue il `pingAll()`, il quale esegue il test di connettività tra tutti gli host nella topologia di rete simulata inviando pacchetti ICMP (ping) da ciascun host a tutti gli altri host e verificando se possono comunicare correttamente. Ne consegue che alla fine di tale operazione, gli switch avranno al loro interno le regole per inoltrare ogni pacchetto che viaggia all'interno della rete e pertanto fornendo il comando `sudo ovs-ofctl dump-flows` relativo a ogni switch OpenFlow verranno visualizzate in output le regole di flusso (flow) configurate su ognuno di essi in maniera completa.

Per visualizzare la topologia realizzata si è utilizzato il tool SDN Narmox Spear, il quale opera tramite il risultato dei comandi dump e links forniti da riga di comando.



Si riporta inoltre la figura che rappresenta la topologia realizzata tramite il tool grafico fornito da Mininet, MiniEdit, nella quale viene messo in evidenza anche il controllore.



Controller base

Il controllore di base implementato garantisce agli switch la capacità di apprendere il MAC address degli host da esso raggiungibili, salvando l'informazione nella MAC address table e di trasferire dunque sul porto corretto i pacchetti ricevuti ed indirizzati verso degli host già noti. Gli switch OpenFlow possono svolgere queste operazioni ricevendo istruzioni da un controllore OpenFlow, come Ryu.

Il metodo costruttore della classe SimpleSwitch13 istanzia la MAC address table, `mac_to_port`, la quale verrà utilizzata per memorizzare i collegamenti tra le porte di ogni switch e i MAC address delle interfacce a cui esse si connettono. Quando il controllore Ryu riceve un messaggio OpenFlow, viene generato un evento corrispondente al messaggio ricevuto, ne consegue la necessità che il controllore implementi delle procedure che gestiscano gli eventi associati ai messaggi che è interessato a ricevere. Le funzioni atte a gestire gli eventi sono precedute da un **decoratore**, il quale fa sì che il corpo della funzione venga eseguito alla generazione

dell'evento e nel momento in cui lo switch si trova in una specifica condizione di funzionamento.

Il primo evento che si attende, una volta completata la procedura di handshake tra lo switch e il controller, è quello detto `EventOFPSwitchFeatures`, il quale viene utilizzato unicamente per dare una tempificazione all'aggiunta della **table-miss flow entry** come regola di base dello switch. Essa ha priorità zero e stabilisce che nel momento in cui uno switch riceve un pacchetto per il quale non è presente ancora la regola d'istramento, questo venga rediretto verso il controller.

Vi è poi una funzione che gestisce l'evento `EventOFPPacketIn`, che si genera nel momento in cui un pacchetto con destinazione non nota viene rediretto verso il controller, dopo che lo switch ha completato l'inizializzazione e si trova in condizioni di funzionamento normali (`MAIN_DISPATCHER`). Il datapath ID viene utilizzato per memorizzare nel dizionario inizialmente definito le rotte relative a ogni porto di uno switch componente la rete. Dal pacchetto ricevuto si ottengono informazioni circa il datapath ID, l'indirizzo sorgente e destinazione MAC, nonché il porto d'ingresso del pacchetto nello switch considerato. Si memorizza pertanto nel dizionario la corrispondenza tra MAC sorgente e porto d'ingresso e, se l'indirizzo di destinazione MAC è noto allo switch considerato, si conosce già il porto su cui inoltrare il pacchetto, altrimenti si procede con il flooding, inoltrando il pacchetto su tutti i porti dello switch. Se la destinazione è nota, si costruisce invece una regola con priorità 1 alla forwarding table dello switch interessato. La funzione finalizzata all'aggiunta di una nuova regola d'inoltro, `add_flow`, riceve come parametri il datapath dello switch a cui la regola fa riferimento, la priorità, i criteri di matching e le azioni da effettuare se questi vengono soddisfatti.

Generazione del traffico e simulazione dell'attacco

Si procede innanzitutto attivando il controllore tramite il comando `ryu-manager` (il controllore attivato in questa fase è il `SimpleSwitch13` descritto al paragrafo precedente) e in un altro terminale si lancia lo script di Python relativo alla topologia. Poiché il codice relativo alla topologia testa la connettività tra ogni coppia di host tramite il comando `pingAll()`, si possono osservare in output le forwarding table relative a ogni switch già complete di tutte le regole di instradamento necessarie.

```

*** Starting the environment
*** Starting controller
Connecting to remote controller at 127.0.0.1:6653
*** Adding hosts
*** Adding switches
*** Adding links
(10.00Mbit) (10.00Mbit) (10.00Mbit) (10.00Mbit) (10.00Mbit) (10.00Mbit) (10.00Mbit) (10.00Mbit) (12.00Mbit) (12.00Mbit) (12.00Mbit) (12.00Mbit) *** Starting network
*** Configuring hosts
h1 h2 h3
*** Starting controller
c1
*** Starting 4 switches
s1 (10.00Mbit) (10.00Mbit) s2 (10.00Mbit) (10.00Mbit) s3 (10.00Mbit) (10.00Mbit) (12.00Mbit) s4 (12.00Mbit) (12.00Mbit) ... (10.00Mbit) (10.00Mbit) (10.00Mbit) (10.00Mbit) (1
0.00Mbit) (10.00Mbit) (12.00Mbit) (12.00Mbit)
*** Testing connectivity (pingAll)
*** Ping: testing ping reachability
h1 -> h2 h3
h2 -> h1 h3
h3 -> h1 h2
*** Results: 0% dropped (6/6 received)
*** Showing rules learned by switches
*Switch s1*
cookie=0x0, duration=0.023s, table=0, n_packets=2, n_bytes=196, priority=1,in_port="s1-eth2",dl_src=00:00:00:00:00:02,dl_dst=00:00:00:00:00:01 actions=output:"s1-eth1"
cookie=0x0, duration=0.022s, table=0, n_packets=1, n_bytes=98, priority=1,in_port="s1-eth1",dl_src=00:00:00:00:00:01,dl_dst=00:00:00:00:00:02 actions=output:"s1-eth2"
cookie=0x0, duration=0.017s, table=0, n_packets=1, n_bytes=98, priority=1,in_port="s1-eth2",dl_src=00:00:00:00:00:03,dl_dst=00:00:00:00:00:01 actions=output:"s1-eth1"
cookie=0x0, duration=0.017s, table=0, n_packets=0, n_bytes=0, priority=1,in_port="s1-eth1",dl_src=00:00:00:00:00:01,dl_dst=00:00:00:00:00:03 actions=output:"s1-eth2"
cookie=0x0, duration=0.143s, table=0, n_packets=7, n_bytes=406, priority=0 actions=CONTROLLER:65535
*Switch s2*
cookie=0x0, duration=0.026s, table=0, n_packets=2, n_bytes=196, priority=1,in_port="s2-eth1",dl_src=00:00:00:00:00:02,dl_dst=00:00:00:00:00:01 actions=output:"s2-eth2"
cookie=0x0, duration=0.023s, table=0, n_packets=1, n_bytes=98, priority=1,in_port="s2-eth2",dl_src=00:00:00:00:00:01,dl_dst=00:00:00:00:00:02 actions=output:"s2-eth1"
cookie=0x0, duration=0.013s, table=0, n_packets=1, n_bytes=98, priority=1,in_port="s2-eth2",dl_src=00:00:00:00:00:03,dl_dst=00:00:00:00:00:02 actions=output:"s2-eth1"
cookie=0x0, duration=0.012s, table=0, n_packets=0, n_bytes=0, priority=1,in_port="s2-eth1",dl_src=00:00:00:00:00:02,dl_dst=00:00:00:00:00:03 actions=output:"s2-eth2"
cookie=0x0, duration=0.121s, table=0, n_packets=9, n_bytes=578, priority=0 actions=CONTROLLER:65535
*Switch s3*
cookie=0x0, duration=0.027s, table=0, n_packets=2, n_bytes=196, priority=1,in_port="s3-eth2",dl_src=00:00:00:00:00:02,dl_dst=00:00:00:00:00:01 actions=output:"s3-eth1"
cookie=0x0, duration=0.025s, table=0, n_packets=1, n_bytes=98, priority=1,in_port="s3-eth1",dl_src=00:00:00:00:00:01,dl_dst=00:00:00:00:00:02 actions=output:"s3-eth2"
cookie=0x0, duration=0.021s, table=0, n_packets=1, n_bytes=98, priority=1,in_port="s3-eth3",dl_src=00:00:00:00:00:03,dl_dst=00:00:00:00:00:01 actions=output:"s3-eth1"
cookie=0x0, duration=0.020s, table=0, n_packets=0, n_bytes=0, priority=1,in_port="s3-eth1",dl_src=00:00:00:00:00:01,dl_dst=00:00:00:00:00:03 actions=output:"s3-eth3"
cookie=0x0, duration=0.015s, table=0, n_packets=1, n_bytes=98, priority=1,in_port="s3-eth3",dl_src=00:00:00:00:00:03,dl_dst=00:00:00:00:00:02 actions=output:"s3-eth2"
cookie=0x0, duration=0.014s, table=0, n_packets=0, n_bytes=0, priority=1,in_port="s3-eth2",dl_src=00:00:00:00:00:02,dl_dst=00:00:00:00:00:03 actions=output:"s3-eth3"
cookie=0x0, duration=0.109s, table=0, n_packets=10, n_bytes=632, priority=0 actions=CONTROLLER:65535
*Switch s4*
cookie=0x0, duration=0.023s, table=0, n_packets=1, n_bytes=98, priority=1,in_port="s4-eth2",dl_src=00:00:00:00:00:03,dl_dst=00:00:00:00:00:01 actions=output:"s4-eth1"
cookie=0x0, duration=0.021s, table=0, n_packets=1, n_bytes=98, priority=1,in_port="s4-eth1",dl_src=00:00:00:00:00:01,dl_dst=00:00:00:00:00:03 actions=output:"s4-eth2"
cookie=0x0, duration=0.017s, table=0, n_packets=1, n_bytes=98, priority=1,in_port="s4-eth2",dl_src=00:00:00:00:00:03,dl_dst=00:00:00:00:00:02 actions=output:"s4-eth1"
cookie=0x0, duration=0.015s, table=0, n_packets=0, n_bytes=0, priority=1,in_port="s4-eth1",dl_src=00:00:00:00:00:02,dl_dst=00:00:00:00:00:03 actions=output:"s4-eth2"
cookie=0x0, duration=0.084s, table=0, n_packets=8, n_bytes=492, priority=0 actions=CONTROLLER:65535
*** Running CLI
*** Starting CLI:
mininet>

```

Con il comando xterm si aprono i terminali degli host coinvolti: due finestre per h3 (server) e una per h1 e h2 (client).

- h1 genera traffico UDP verso h3
- h2 genera traffico TCP verso h3

La banda tra i due switch interni, s3 e s4, e tra s4 e l'host server h3 è settata a un valore di 12 Mbit/s, mentre gli altri link sono settati a un valore di banda pari a 10 Mbit/s; si è infatti verificato che questa scelta consente di sostenere due comunicazioni con throughput pari a 5 Mbit/s: quella tra l'host client h1 che utilizza il protocollo di trasporto UDP e il server h3 e quella tra l'host client h2 che utilizza il protocollo di trasporto TCP e il server h3. Quanto detto si verifica tramite il comando iperf, un tool per la generazione di traffico in una rete, tipicamente utilizzato avviando un primo processo iperf in server mode sull'host che riceve il traffico ed eseguendo un secondo processo in client mode sull'host che invia il traffico.

```

# Su h3 — server TCP sulla porta 5001
iperf -s -i 1 | tee throughput2_normal.txt

# Su h3 — server UDP sulla porta 5003
iperf -s -u -p 5003 -i 1 | tee throughput1_normal.txt

```

```
"Node: h1"
root@ubuntu:/home/ubuntu/Desktop/mininet# iperf -c 10.0.0.3 -u -p 5003 -b 5M -t 60
-----
Client connecting to 10.0.0.3, UDP port 5003
Sending 1470 byte datagrams, IPG target: 2243.04 us (kalman adjust)
UDP buffer size: 208 KByte (default)
-----
[ 1] local 10.0.0.1 port 56653 connected with 10.0.0.3 port 5003
[ ID] Interval      Transfer    Bandwidth
[ 1] 0.0000-60.0037 sec 37.5 MBytes 5.24 Mbits/sec
[ 1] Sent 26754 datagrams
[ 1] Server Report:
[ ID] Interval      Transfer    Bandwidth      Jitter  Lost/Total Datagrams
[ 1] 0.0000-60.0031 sec 37.5 MBytes 5.24 Mbits/sec 0.020 ms 0/26753 (0%)
root@ubuntu:/home/ubuntu/Desktop/mininet#

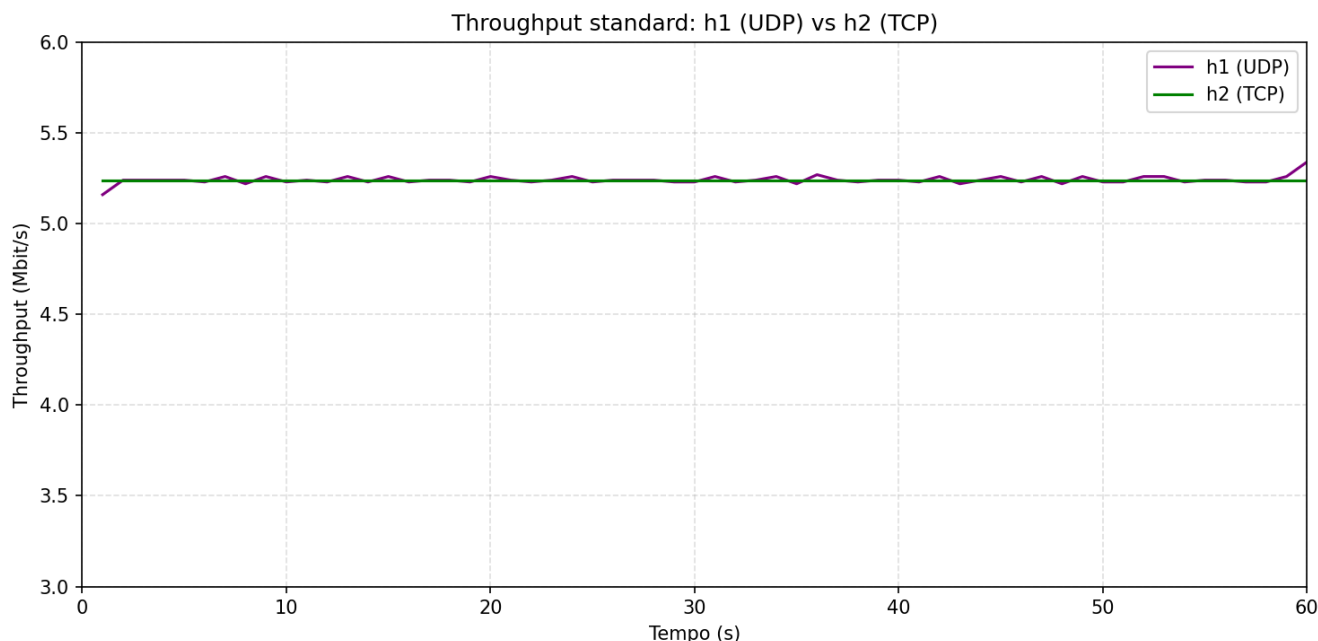
"Node: h3"
root@ubuntu:/home/ubuntu/Desktop/mininet# iperf -s -u -p 5003 -i 1 | tee throughput1_normal.txt
-----
Server listening on UDP port 5003
UDP buffer size: 208 KByte (default)
-----
[ 1] local 10.0.0.3 port 5003 connected with 10.0.0.1 port 56653
[ ID] Interval      Transfer    Bandwidth      Jitter  Lost/Total Datagrams
[ 1] 0.0000-1.0000 sec 630 KBytes 5.16 Mbits/sec 1.725 ms 0/439 (0%)
[ 1] 1.0000-2.0000 sec 640 KBytes 5.24 Mbits/sec 1.959 ms 0/446 (0%)
[ 1] 2.0000-3.0000 sec 640 KBytes 5.24 Mbits/sec 1.738 ms 0/446 (0%)
[ 1] 3.0000-4.0000 sec 640 KBytes 5.24 Mbits/sec 1.641 ms 0/446 (0%)
[ 1] 4.0000-5.0000 sec 640 KBytes 5.24 Mbits/sec 1.696 ms 0/446 (0%)
[ 1] 5.0000-6.0000 sec 639 KBytes 5.23 Mbits/sec 1.755 ms 0/445 (0%)
[ 1] 6.0000-7.0000 sec 642 KBytes 5.26 Mbits/sec 1.671 ms 0/447 (0%)
[ 1] 7.0000-8.0000 sec 637 KBytes 5.22 Mbits/sec 1.840 ms 0/444 (0%)
[ 1] 8.0000-9.0000 sec 642 KBytes 5.26 Mbits/sec 1.676 ms 0/447 (0%)
[ 1] 9.0000-10.0000 sec 639 KBytes 5.23 Mbits/sec 1.854 ms 0/445 (0%)
[ 1] 10.0000-11.0000 sec 640 KBytes 5.24 Mbits/sec 1.849 ms 0/446 (0%)
[ 1] 11.0000-12.0000 sec 639 KBytes 5.23 Mbits/sec 1.951 ms 0/445 (0%)
[ 1] 12.0000-13.0000 sec 642 KBytes 5.26 Mbits/sec 1.915 ms 0/447 (0%)
[ 1] 13.0000-14.0000 sec 639 KBytes 5.23 Mbits/sec 1.699 ms 0/445 (0%)
[ 1] 14.0000-15.0000 sec 642 KBytes 5.26 Mbits/sec 1.911 ms 0/447 (0%)
[ 1] 15.0000-16.0000 sec 639 KBytes 5.23 Mbits/sec 1.912 ms 0/445 (0%)

"Node: h2"
root@ubuntu:/home/ubuntu/Desktop/mininet# iperf -c 10.0.0.3 -b 5M -t 60
-----
Client connecting to 10.0.0.3, TCP port 5001
TCP window size: 85.3 KByte (default)
-----
[ 1] local 10.0.0.2 port 46398 connected with 10.0.0.3 port 5001
[ ID] Interval      Transfer    Bandwidth
[ 1] 0.0000-60.1566 sec 37.6 MBytes 5.25 Mbits/sec
root@ubuntu:/home/ubuntu/Desktop/mininet#

"Node: h3"
root@ubuntu:/home/ubuntu/Desktop/mininet# iperf -s -i 1 | tee throughput2_normal.txt
-----
Server listening on TCP port 5001
TCP window size: 85.3 KByte (default)
-----
[ 1] local 10.0.0.3 port 5001 connected with 10.0.0.2 port 46398
[ ID] Interval      Transfer    Bandwidth
[ 1] 0.0000-1.0000 sec 640 KBytes 5.24 Mbits/sec
[ 1] 1.0000-2.0000 sec 640 KBytes 5.24 Mbits/sec
[ 1] 2.0000-3.0000 sec 640 KBytes 5.24 Mbits/sec
[ 1] 3.0000-4.0000 sec 640 KBytes 5.24 Mbits/sec
[ 1] 4.0000-5.0000 sec 640 KBytes 5.24 Mbits/sec
[ 1] 5.0000-6.0000 sec 640 KBytes 5.24 Mbits/sec
[ 1] 6.0000-7.0000 sec 640 KBytes 5.24 Mbits/sec
[ 1] 7.0000-8.0000 sec 640 KBytes 5.24 Mbits/sec
[ 1] 8.0000-9.0000 sec 640 KBytes 5.24 Mbits/sec
[ 1] 9.0000-10.0000 sec 640 KBytes 5.24 Mbits/sec
[ 1] 10.0000-11.0000 sec 640 KBytes 5.24 Mbits/sec
[ 1] 11.0000-12.0000 sec 640 KBytes 5.24 Mbits/sec
[ 1] 12.0000-13.0000 sec 640 KBytes 5.24 Mbits/sec
[ 1] 13.0000-14.0000 sec 640 KBytes 5.24 Mbits/sec
[ 1] 14.0000-15.0000 sec 640 KBytes 5.24 Mbits/sec
[ 1] 15.0000-16.0000 sec 640 KBytes 5.24 Mbits/sec
```

Come mostrato in figura, si attiva un server iperf TCP (standard) sul porto 5001 (standard) di h3 e un server iperf UDP sul porto 5003 di h3: in entrambi i casi si memorizzano i valori relativi alle performance in un file di testo con cadenza pari a 1 secondo; si attivano dunque i client h1 e h2 comunicanti con h3, il quale è identificato dal suo indirizzo IP 10.0.0.3 e dal suo porto (laddove non sia quello standard), e si riserva a ciascun flusso una banda di 5 Mbit/s.

Si può notare come i throughput rimangano pressochè stabili in un piccolo intorno di 5 Mbit/s, pertanto le performance in questa fase non subiscono alcuna degradazione, poiché ogni client sfrutta correttamente la banda che gli viene assegnata per la comunicazione.



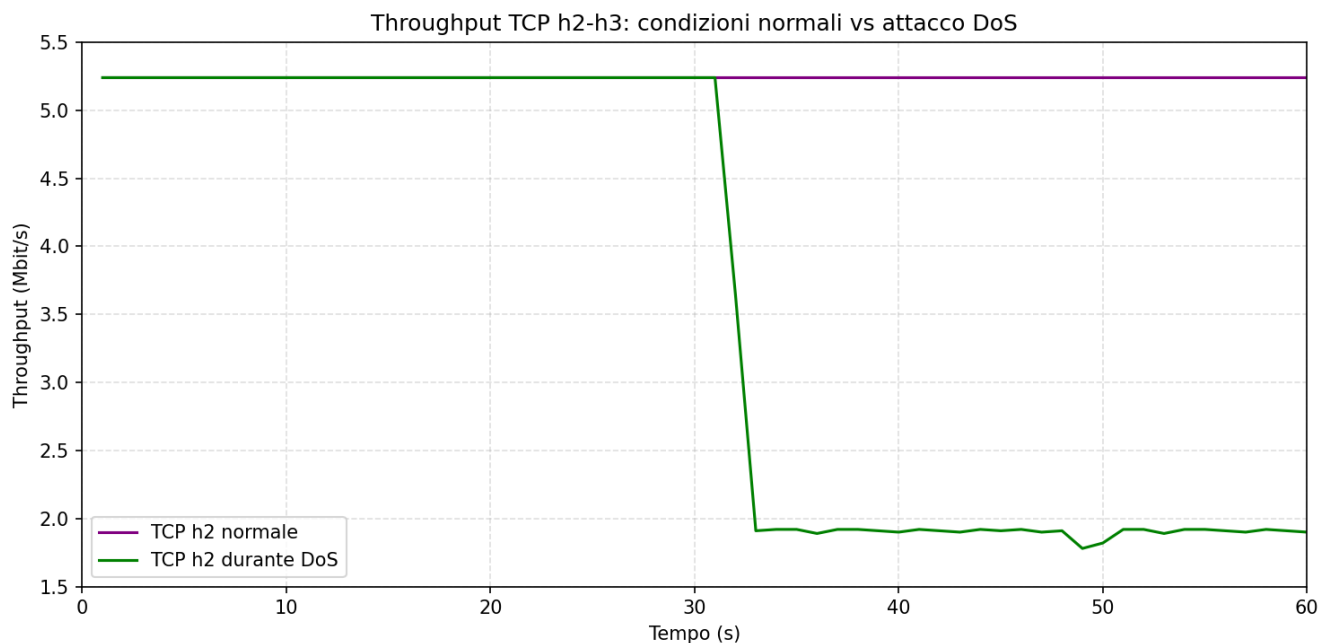
Generazione attacco DoS e valutazione performance

Si vuole ora mostrare come le prestazioni della comunicazione TCP tra h2 e h3 degradino notevolmente nel momento in cui la rete è sottoposta a un attacco Denial of Service (DoS) di tipo UDP flooding da parte dell'host h1, il quale inonda il server h3 con una quantità di traffico che non riesce a gestire causando la congestione della rete e rendendo il server inaccessibile agli utenti legittimi.

Per simulare questo scenario, rispetto al passo precedente si modifica solamente il comando iperf lanciato da h1, il quale adesso genera un flusso dati di banda 10 Mbit/s con 6 connessioni parallele verso il server:

```
iperf -c 10.0.0.3 -u -b 10M -P 6
```

Si riporta dunque il grafico rappresentante il throughput di h3 relativo alla connessione TCP con h2 in condizioni normali (linea viola) e in presenza di attacco DoS (linea verde): è evidente il degrado prestazionale causato.



Oltre al valore relativo al throughput, si vogliono valutare anche ulteriori parametri relativi alla Quality of Service (QoS), in particolare il **delay** e il **jitter**. Per fare ciò si è utilizzato D-ITG (Distributed Internet Traffic Generator), una piattaforma capace di generare traffico IPv4 e al contempo è un tool capace di misurare le più comuni metriche di performance della rete a livello pacchetto. Si procede avviando ITGRecv nell'host ricevitore h3, ovvero la componente responsabile di ricevere traffici multipli derivanti da una o più istanze di ITGSend; quest'ultima è infatti avviata da h1 e h2 istaurando una comunicazione verso h3 tramite la specificazione dell'indirizzo IP destinazione, del numero di pacchetti al secondo da inviare con inter-departure time costante e della dimensione in byte dei pacchetti. Anche in questo caso le performance vengono salvate in un file, è pertanto necessario avviare un'istanza di ITGLog da h3, la quale riceve e memorizza le informazioni inviate da ITGSend e ITGRecv.

```
# Su h3 – avvio ricevitore e logger
```

```
ITGRecv
```

```
ITGLog
```

```
# Traffico inviato da h1 (UDP, default)
```

```
ITGSend -a 10.0.0.3 -C 1000 -c 625 -t 60000 -x recv1
```

```
# Traffico inviato da h2 (TCP)
```

```
ITGSend -a 10.0.0.3 -T TCP -C 1000 -c 625 -t 60000 -x recv2
```

Si riportano in figura i risultati complessivi di quest'esecuzione relativi alla comunicazione tra h2 e h3 generati tramite il comando ITGDec lanciato nel terminale del ricevitore relativamente al file di log, esso è infatti la componente responsabile della decodifica e dell'analisi dei file di log memorizzati. Si può notare come il throughput medio sia, coerentemente con quanto avviene con iperf, pari all'incirca a 5 Mbit/s, inoltre si vede come i valori di delay e di jitter siano notevolmente bassi.

```

root@ubuntu:/home/ubuntu/Desktop/mininet# ITGDec recv2
ITGDec version 2.8.1 (r1023)
Compile-time options: sctp dccp bursty multiport
/-----
Flow number: 1
From 10.0.0.2:47606
To   10.0.0.3:8999
-----
Total time           = 60.000497 s
Total packets        = 55804
Minimum delay        = 0.000010 s
Maximum delay        = 0.029491 s
Average delay        = 0.000478 s
Average jitter       = 0.000565 s
Delay standard deviation = 0.000831 s
Bytes received       = 34877500
Average bitrate      = 4650.294813 Kbit/s
Average packet rate  = 930.058963 pkt/s
Packets dropped      = 0 (0.00 %)
Average loss-burst size = 0.000000 pkt
-----

***** TOTAL RESULTS *****
-----
Number of flows      = 1
Total time           = 60.000497 s
Total packets        = 55804
Minimum delay        = 0.000010 s
Maximum delay        = 0.029491 s
Average delay        = 0.000478 s
Average jitter       = 0.000565 s
Delay standard deviation = 0.000831 s
Bytes received       = 34877500
Average bitrate      = 4650.294813 Kbit/s
Average packet rate  = 930.058963 pkt/s
Packets dropped      = 0 (0.00 %)
Average loss-burst size = 0 pkt
Error lines          = 0
-----

```

Nella seconda esecuzione, invece, si vuole generare l'attacco DoS da parte di h1: ciò viene fatto modificando i comandi di d-itg solamente dal terminale di h1. Quello che si va a fare è generare 6 flussi paralleli, tutti con le stesse caratteristiche, tramite uno script, che viene poi lanciato con il comando ITGSend.

Su h3 – avvio ricevitore e logger

ITGRecv

ITGLog

Traffico inviato da h1 (UDP, default)

ITGSend script_file

Traffico inviato da h2 (TCP)

```
ITGSend -a 10.0.0.3 -T TCP -C 1000 -c 625 -t 60000 -x attack2
```

Contenuto script_file

```
-a 10.0.0.3 -rp 10001 -C 10000000 -c 100000 -T UDP -t 60000
-a 10.0.0.3 -rp 10002 -C 10000000 -c 100000 -T UDP -t 60000
-a 10.0.0.3 -rp 10003 -C 10000000 -c 100000 -T UDP -t 60000
-a 10.0.0.3 -rp 10004 -C 10000000 -c 100000 -T UDP -t 60000
-a 10.0.0.3 -rp 10005 -C 10000000 -c 100000 -T UDP -t 60000
-a 10.0.0.3 -rp 10006 -C 10000000 -c 100000 -T UDP -t 60000
```

Si riportano dunque i risultati delle performance di h2 in questo secondo caso, in cui si può subito notare come il throughput medio si sia abbassato e i valori di delay e jitter siano invece aumentati.

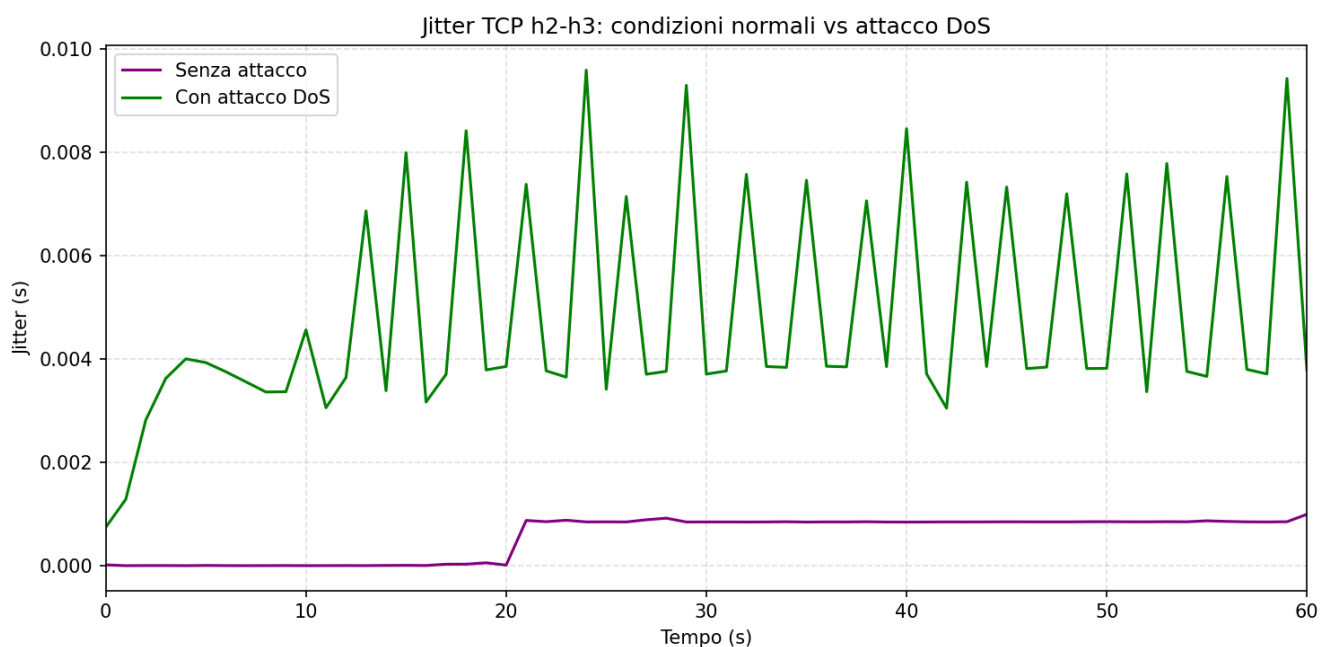
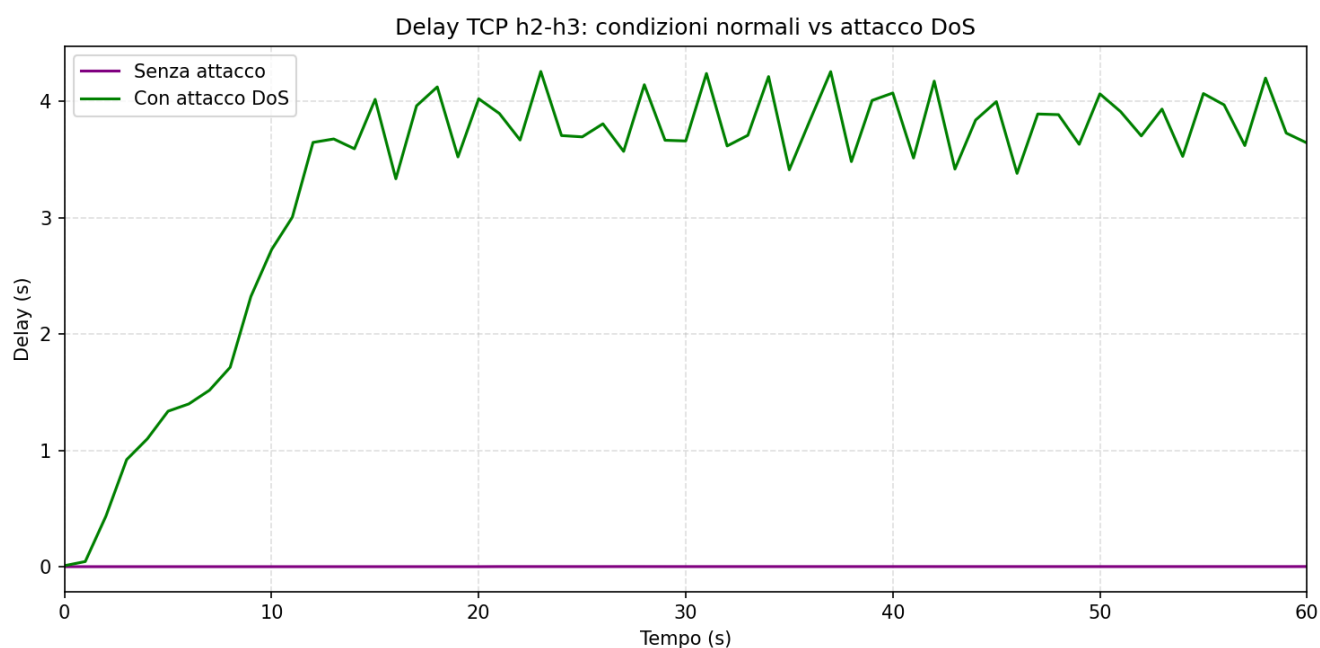
```
root@arcade-Standard-PC-Q35-ICH9-2009:/home/arcade/Desktop/mininet# ITGDec attack2
ITGDec version 2.8.1 (r1023)
Compile-time options: sctp dccp bursty multiport
-----
Flow number: 1
From 10.0.0.2:40258
To 10.0.0.3:8999
-----
Total time = 62.839763 s
Total packets = 26279
Minimum delay = 0.000219 s
Maximum delay = 4.954453 s
Average delay = 3.068978 s
Average jitter = 0.004470 s
Delay standard deviation = 1.372544 s
Bytes received = 16424375
Average bitrate = 2090.953144 Kbit/s
Average packet rate = 418.190629 pkt/s
Packets dropped = 0 (0.00 %)
Average loss-burst size = 0.000000 pkt
-----
***** TOTAL RESULTS *****
-----
Number of flows = 1
Total time = 62.839763 s
Total packets = 26279
Minimum delay = 0.000219 s
Maximum delay = 4.954453 s
Average delay = 3.068978 s
Average jitter = 0.004470 s
Delay standard deviation = 1.372544 s
Bytes received = 16424375
Average bitrate = 2090.953144 Kbit/s
Average packet rate = 418.190629 pkt/s
Packets dropped = 0 (0.00 %)
Average loss-burst size = 0 pkt
Error lines = 0
-----
```

Si è dunque graficato l'andamento del delay e del jitter nella prima simulazione senza attacco e nella seconda simulazione con attacco DoS, ricavando tali valori dai file di log tramite i comandi

```
ITGDec <logfile> -d 1000 <logname.txt> # delay per secondo
```

```
ITGDec <logfile> -j 1000 <logname.txt> # jitter per secondo
```

che forniscono un file .dat contenente rispettivamente delay e jitter per secondo. Si può notare come senza attacco, delay e jitter restano prossimi allo zero; durante l'attacco DoS il delay raggiunge picchi di circa 4 secondi, con un lieve peggioramento del jitter.



Sistema di monitoraggio e blocco dell'attacco

Il controllore di base definito viene dunque esteso inserendo delle funzioni atte a monitorare lo stato della rete e a bloccare eventuali attacchi rilevati dall'analisi dei dati raccolti. La procedura di monitoraggio distingue ogni switch e ogni porta ad essi associata, valutando la quantità di dati che ogni porta riceve e invia durante un certo intervallo di tempo stabilito. Si è scelto di analizzare le statistiche della rete ogni dieci secondi, in quanto in questo modo si riesce a rilevare un attacco in maniera tempestiva riducendo i tempi in cui il servizio di rete è non disponibile.

Si implementa la classe `SimpleMonitor13` estendendo la classe `SimpleSwitch13`, dalla quale eredita tutti i metodi. Il costruttore, oltre a invocare il costruttore della classe `SimpleSwitch13`, crea un thread che esegue la funzione periodica di `_monitor`, la quale ogni dieci secondi invia una richiesta a ogni switch, identificato dal datapath, di fornire indicazioni sul proprio stato. Per mantenere una lista di tutti gli switch connessi al controller, ai quali inviare la richiesta di stato periodica, si utilizza l'evento `EventOFPSStateChange`, il quale viene generato nel momento in cui uno switch si connette o si disconnette dal controller. Se al generarsi dell'evento lo stato dello switch che ha inviato il messaggio risulta in modalità `MAIN_DISPATCHER`, allora si memorizza il suo datapath, se invece risulta in modalità `DEAD_DISPATCHER`, il suo datapath viene rimosso dalla lista.

La funzione invocata da `_monitor` per far sì che gli switch forniscano le statistiche relative ai propri flussi e porti è `_request_stats`, la quale inoltra agli switch un messaggio `OFPPFlowsStatsRequest` e un messaggio `OFPPortStatsRequest`. La risposta a quest'ultimo viene utilizzata per la detection dell'attacco, in quanto fornisce delle statistiche relative a ogni porto di uno switch.

La funzione che gestisce l'evento di risposta `EventOFPPortStatsReply` è chiamata `_port_stats_reply` ed è in essa che viene inserito anche il codice relativo al blocco dell'attacco eventualmente riscontrato. Si calcola, infatti, il throughput relativo a ogni porto sfruttando il valore di byte che esso ha ricevuto nell'intervallo di monitoraggio. A ogni invocazione della funzione si salva il valore `rx_bytes` in maniera tale che alla successiva attivazione effettuando la sottrazione tra la nuova quantità di byte ricevuta e la precedente e dividendo per l'intervallo di tempo di monitoraggio scelto, si possa ottenere il throughput, relativo a uno specifico switch e a un suo specifico porto.

Il throughput così ottenuto va confrontato con un valore di soglia per stabilire se sia in corso o meno un attacco e settare in tal caso una variabile di allarme che inneschi il meccanismo di blocco. Poiché ogni switch gestisce una quantità di traffico differente, anche le soglie da valutare sono state differenziate. In particolare per gli switch `s1` e `s2`, i quali convogliano il traffico di un solo host, la soglia è stata settata pari a 6 Mb/s,

pari a 750000 byte/s, in accordo a quanto stabilito nei paragrafi precedenti, nei quali si è scelto di far sì che gli host h1 e h2 possano inoltrare un traffico pari a 5 Mb/s verso l'host h3. Per quanto concerne, invece, la soglia per s3 e s4, questa è stata settata pari a 8.6 Mb/s, ovvero 1075000 byte/s, poiché il throughput sui link di questi switch supera tale valore solamente nel momento in cui uno tra i client h1 e h2 supera la banda che gli è riservata.

Per quanto concerne la logica di blocco, quello che si fa è aggiungere una regola di drop ai pacchetti che giungono alla porta dello switch per la quale si è misurato un throughput superiore alla soglia tramite il metodo `_block_attack`, il quale è infatti responsabile dell'aggiunta di tale regola alle entry della forwarding table dello switch sotto esame. È importante notare che tale regola deve avere priorità 2, poiché le regole di base definite dal controller principale hanno tutte priorità 1 e il comportamento desiderato è tale per cui se è presente una regola di drop per uno specifico porto d'ingresso, questa sia predominante sulle altre regole presenti.

Quando un porto viene bloccato, esso viene aggiunto, assieme al datapath id dello switch a cui appartiene, all'interno di un dizionario, che memorizza gli switch e i relativi porti bloccati, detto `blocked_list`; al contempo, viene settato un valore di timer associato al porto bloccato pari a 10 s, il quale sarà decrementato nell'iterazione di monitoraggio successiva. Scopo di tale timer è individuare la sorgente dell'attacco non bloccando tutta la rete e dunque consentendo agli utenti legittimi di continuare la propria comunicazione. Se, infatti, dopo 10 secondi, il throughput del link bloccato risulta già sotto soglia, se ne deduce che non era esso la sorgente di attacco e pertanto si procede rimuovendo la regola di drop tramite la funzione `remove_table_flow`; viceversa, se dopo 10 secondi dal blocco, il throughput del link è ancora sopra la soglia, esso viene individuato come sorgente di attacco: se ne prolunga il blocco per ulteriori 90 secondi e si sbloccano tutti gli altri link ancora eventualmente presenti nella `blocked_list`.

Si noti che dopo l'individuazione della sorgente dell'attacco e il conseguente sblocco degli ulteriori link bloccati, il sistema di monitoraggio viene messo in pausa per 50 secondi: tale intervallo di tempo è necessario a far sì che la connessione tra h2 e h3 ritorni in condizioni di funzionamento normali, poiché dopo lo sblocco del link tra s2 e s3 e tra s3 e s4, il traffico TCP supera il valore di 6 Mb/s per un piccolo intervallo di tempo, in quanto i pacchetti non trasmessi durante il periodo di blocco devono essere recuperati.

Si riportano le forwarding table degli switch, una volta che l'attaccante è stato individuato e bloccato e la rete è ritornata al proprio funzionamento di base: si può osservare come solamente lo switch s1 abbia tra le regole quella di drop, relativa al porto d'ingresso 1, al quale è appunto collegato l'host client h1, mentre le altre forwarding table risultano invariate.

```

ubuntu@ubuntu:~/Desktop/mininet$ sudo ovs-ofctl dump-flows s1
cookie=0x0, duration=138.336s, table=0, n_packets=25343, n_bytes=38268664, priority=2,in_port="s1-eth1" actions=drop
cookie=0x0, duration=215.981s, table=0, n_packets=3, n_bytes=238, priority=1,in_port="s1-eth2",dl_src=00:00:00:00:02,dl_dst=00:00:00:00:01 actions=output:"s1-eth1"
cookie=0x0, duration=215.980s, table=0, n_packets=2, n_bytes=140, priority=1,in_port="s1-eth1",dl_src=00:00:00:00:01,dl_dst=00:00:00:00:02 actions=output:"s1-eth2"
cookie=0x0, duration=215.976s, table=0, n_packets=13, n_bytes=1426, priority=1,in_port="s1-eth2",dl_src=00:00:00:00:03,dl_dst=00:00:00:00:01 actions=output:"s1-eth1"
cookie=0x0, duration=215.975s, table=0, n_packets=7480, n_bytes=11305406, priority=1,in_port="s1-eth1",dl_src=00:00:00:00:01,dl_dst=00:00:00:00:03 actions=output:"s1-eth2"
cookie=0x0, duration=216.111s, table=0, n_packets=112, n_bytes=11732, priority=0 actions=CONTROLLER:65535
ubuntu@ubuntu:~/Desktop/mininet$ sudo ovs-ofctl dump-flows s2
cookie=0x0, duration=217.595s, table=0, n_packets=3, n_bytes=238, priority=1,in_port="s2-eth1",dl_src=00:00:00:00:02,dl_dst=00:00:00:00:01 actions=output:"s2-eth2"
cookie=0x0, duration=217.593s, table=0, n_packets=2, n_bytes=140, priority=1,in_port="s2-eth2",dl_src=00:00:00:00:01,dl_dst=00:00:00:00:02 actions=output:"s2-eth1"
cookie=0x0, duration=217.576s, table=0, n_packets=12462, n_bytes=822988, priority=1,in_port="s2-eth2",dl_src=00:00:00:00:03,dl_dst=00:00:00:00:02 actions=output:"s2-eth1"
cookie=0x0, duration=217.572s, table=0, n_packets=13109, n_bytes=57442686, priority=1,in_port="s2-eth1",dl_src=00:00:00:00:02,dl_dst=00:00:00:00:03 actions=output:"s2-eth2"
cookie=0x0, duration=217.703s, table=0, n_packets=112, n_bytes=11732, priority=0 actions=CONTROLLER:65535
ubuntu@ubuntu:~/Desktop/mininet$ sudo ovs-ofctl dump-flows s3
cookie=0x0, duration=219.640s, table=0, n_packets=3, n_bytes=238, priority=1,in_port="s3-eth2",dl_src=00:00:00:00:02,dl_dst=00:00:00:00:01 actions=output:"s3-eth1"
cookie=0x0, duration=219.638s, table=0, n_packets=2, n_bytes=140, priority=1,in_port="s3-eth1",dl_src=00:00:00:00:01,dl_dst=00:00:00:00:02 actions=output:"s3-eth2"
cookie=0x0, duration=219.634s, table=0, n_packets=13, n_bytes=1426, priority=1,in_port="s3-eth3",dl_src=00:00:00:00:03,dl_dst=00:00:00:00:01 actions=output:"s3-eth1"
cookie=0x0, duration=219.633s, table=0, n_packets=7479, n_bytes=11303894, priority=1,in_port="s3-eth1",dl_src=00:00:00:00:01,dl_dst=00:00:00:00:03 actions=output:"s3-eth3"
cookie=0x0, duration=219.622s, table=0, n_packets=12462, n_bytes=822988, priority=1,in_port="s3-eth3",dl_src=00:00:00:00:03,dl_dst=00:00:00:00:02 actions=output:"s3-eth2"
cookie=0x0, duration=219.617s, table=0, n_packets=13109, n_bytes=57442686, priority=1,in_port="s3-eth2",dl_src=00:00:00:00:02,dl_dst=00:00:00:00:03 actions=output:"s3-eth3"
cookie=0x0, duration=219.729s, table=0, n_packets=113, n_bytes=11762, priority=0 actions=CONTROLLER:65535
ubuntu@ubuntu:~/Desktop/mininet$ sudo ovs-ofctl dump-flows s4
cookie=0x0, duration=221.492s, table=0, n_packets=13, n_bytes=1426, priority=1,in_port="s4-eth2",dl_src=00:00:00:00:03,dl_dst=00:00:00:00:01 actions=output:"s4-eth1"
cookie=0x0, duration=221.490s, table=0, n_packets=7164, n_bytes=10827614, priority=1,in_port="s4-eth1",dl_src=00:00:00:00:01,dl_dst=00:00:00:00:03 actions=output:"s4-eth2"
cookie=0x0, duration=221.481s, table=0, n_packets=12462, n_bytes=822988, priority=1,in_port="s4-eth2",dl_src=00:00:00:00:03,dl_dst=00:00:00:00:02 actions=output:"s4-eth1"
cookie=0x0, duration=221.475s, table=0, n_packets=13046, n_bytes=57269112, priority=1,in_port="s4-eth1",dl_src=00:00:00:00:02,dl_dst=00:00:00:00:03 actions=output:"s4-eth2"
cookie=0x0, duration=221.558s, table=0, n_packets=111, n_bytes=11622, priority=0 actions=CONTROLLER:65535

```

Il sistema di monitoraggio e blocco dell'attacco implementato tiene in conto due aspetti essenziali: il primo riguarda la possibilità di sbloccare l'attaccante dopo un certo intervallo di tempo, il secondo concerne la formulazione di una soluzione che vada a impattare il meno possibile sulle performance degli utenti legittimi. Il primo punto viene risolto impostando un timer pari a 90 secondi una volta individuata la fonte di attacco; allo scadere del tempo settato, se il link collegato all'attaccante non supera più la threshold stabilita, la relativa regola di drop viene rimossa dalla forwarding table dello switch di riferimento, consentendo all'host di avviare una nuova comunicazione.

Il secondo punto viene risolto tramite dei meccanismi che mirano a individuare il punto di inizio della generazione del traffico anomalo nella topologia della rete, così da bloccare unicamente il link che collega l'attaccante allo switch e da impattare sugli altri host solamente per il tempo necessario alla convergenza. L'algoritmo implementato in un primo momento blocca tutti i link che superano la threshold definita, ma se dopo 10 secondi essi non sperimentano più l'allarme, se ne deduce che non costituiscono la fonte di attacco e vengano pertanto sbloccati, ovvero viene rimossa l'entry contenente la regola di drop nella forwarding table relativa.

Valutazione performance con sistema di monitoraggio e blocco

Si vuole dunque mostrare come tale sistema di monitoraggio e blocco dell'attacco, con cui è stato esteso il controllore di base, faccia sì che le performance della comunicazione TCP tra h2 e h3 non subiscano la medesima degradazione riscontrata, a seguito dell'attacco Dos di h1 verso h3, nel caso in cui esso non era stato introdotto. La generazione del traffico avviene come fatto precedentemente, così da confrontare i risultati ottenuti:

- **Senza mitigazione:** il throughput è inizialmente intorno al valore nominale di 5 Mb/s per poi crollare a 2 Mb/s per tutta la durata dell'attacco.

- **Con mitigazione attiva:** dopo 10s dall'inizio dell'attacco, il throughput crolla brevemente a 2 Mb/s (10s di blocco per l'identificazione della sorgente), poi oscilla temporaneamente tra 9 e 10 Mb/s (fase di recupero dei pacchetti accumulati), per poi ristabilizzarsi intorno ai 5 Mb/s previsti.

